EXPERIMENT_NO.8

In [1]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from google.colab.patches import cv2_imshow
import os
import time
import math

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [11]:

```
uploaded = files.upload()

video_name = list(uploaded.keys())[0]

print("Uploaded video:", video_name)

cap = cv2.VideoCapture(video_name)

if not cap.isOpened():
    print("Error: Video could not be opened.")
else:
    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    print("Video opened successfully!")
    print("FPS:", fps)
    print("Width:", width)
    print("Height:", height)
    print("Total Frames:", total_frames)

cap.release()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving 853762-hd_1920_1080_24fps.mp4 to 853762-hd_1920_1080_24fps.mp4
Uploaded video: 853762-hd_1920_1080_24fps.mp4
Video opened successfully!
FPS: 23.976023976023978
Width: 1920
Height: 1080
Total Frames: 313
```

In [14]:

```
import cv2
import numpy as np
import math
import time

cap = cv2.VideoCapture(video_name)

# Shi-Tomasi corner detection parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

# Read first frame
```

```
ret, old_frame = cap.read()

if not ret:
    print("Could not read first frame.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    # Detect feature points using Shi-Tomasi
    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

    # Mask for drawing trajectories
    mask = np.zeros_like(old_frame)

    # Output video
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(
        'lucas_kanade_output.mp4',
        fourcc,
        20.0,
        (old_frame.shape[1], old_frame.shape[0])
    )

    frame_count = 0
    total_magnitude = 0
    total_direction = 0
    magnitude_count = 0

    start_time = time.time()

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Calculate Optical Flow
        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is not None:

            # Select successfully tracked points
            good_new = p1[st == 1]
            good_old = p0[st == 1]

            magnitudes = []
            directions = []

            for new, old in zip(good_new, good_old):

                x_new, y_new = new.ravel()
                x_old, y_old = old.ravel()

                dx = x_new - x_old
                dy = y_new - y_old

                magnitude = math.sqrt(dx**2 + dy**2)

                # Direction in degrees
                angle = math.degrees(math.atan2(dy, dx))

                magnitudes.append(magnitude)
                directions.append(angle)

                # Draw trajectory
```

```
                mask = cv2.line(
                    mask,
                    (int(x_new), int(y_new)),
                    (int(x_old), int(y_old)),
                    (0, 255, 0),
                    2
                )

                # Draw current point
                frame = cv2.circle(
                    frame,
                    (int(x_new), int(y_new)),
                    4,
                    (0, 0, 255),
                    -1
                )

                # Draw motion vector
                frame = cv2.arrowedLine(
                    frame,
                    (int(x_old), int(y_old)),
                    (int(x_new), int(y_new)),
                    (255, 0, 0),
                    2
                )

            if len(magnitudes) > 0:

                avg_magnitude = np.mean(magnitudes)
                avg_direction = np.mean(directions)

                total_magnitude += avg_magnitude
                total_direction += avg_direction
                magnitude_count += 1

                # Display values
                cv2.putText(
                    frame,
                    f"Avg Motion: {avg_magnitude:.2f} pixels",
                    (20, 30),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.7,
                    (0, 255, 255),
                    2
                )

                cv2.putText(
                    frame,
                    f"Direction: {avg_direction:.2f} degrees",
                    (20, 60),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.7,
                    (0, 255, 255),
                    2
                )

            # Update points
            p0 = good_new.reshape(-1, 1, 2)

        # Combine trajectory mask and current frame
        output_frame = cv2.add(frame, mask)

        out.write(output_frame)

        frame_count += 1

        old_gray = frame_gray.copy()

end_time = time.time()

cap.release()
out.release()

processing_time = end_time - start_time

print("Lucas-Kanade Optical Flow completed!")
print("Frames processed:", frame_count)
```

```
    print("Processing time:", round(processing_time, 2), "seconds")

    if magnitude_count > 0:
        print("Average displacement:",
              round(total_magnitude / magnitude_count, 2),
              "pixels")

        print("Average direction:",
              round(total_direction / magnitude_count, 2),
              "degrees")
```

```
Lucas-Kanade Optical Flow completed!
Frames processed: 312
Processing time: 14.6 seconds
Average displacement: 1.48 pixels
Average direction: 5.09 degrees
```

In [13]:

```
files.download('lucas_kanade_output.mp4')
```

```
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
```

In [16]:

```
cap = cv2.VideoCapture(video_name)

ret, frame1 = cap.read()

if not ret:
    print("Video read nahi ho rahi!")
else:
    frame1 = cv2.resize(frame1, (640, 360))
    prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)

    print("Video ready for Farneback Optical Flow")
```

```
Video ready for Farneback Optical Flow
```

In [17]:

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')

out = cv2.VideoWriter(
    'farneback_output.mp4',
    fourcc,
    20.0,
    (640, 360)
)

frame_count = 0
total_magnitude = 0

print("Output setup complete!")
```

```
Output setup complete!
```

In [18]:

```
while True:

    # Read next frame
    ret, frame2 = cap.read()

    if not ret:
        break

    # Resize
    frame2 = cv2.resize(frame2, (640, 360))
    next_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    # Fast Farneback
    flow = cv2.calcOpticalFlowFarneback(
        prvs,
        next_gray,
        None,
        0.5,
        2,
        8,
        2,
        3,
```

```
        1.1,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    avg_magnitude = np.mean(magnitude)
    total_magnitude += avg_magnitude

    # Optical Flow visualization
    hsv = np.zeros_like(frame2)

    hsv[..., 1] = 255
    hsv[..., 0] = angle * 180 / np.pi / 2

    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    result = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.putText(
        result,
        f"Motion: {avg_magnitude:.2f}",
        (20, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    out.write(result)

    prvs = next_gray
    frame_count += 1

    # Progress
    if frame_count % 20 == 0:
        print("Processed frames:", frame_count)

print("Processing complete!")
```

```
Processed frames: 20
Processed frames: 40
Processed frames: 60
Processed frames: 80
Processed frames: 100
Processed frames: 120
Processed frames: 140
Processed frames: 160
Processed frames: 180
Processed frames: 200
Processed frames: 220
Processed frames: 240
Processed frames: 260
Processed frames: 280
Processed frames: 300
Processing complete!
```

In [19]:

```
cap.release()
out.release()

print("Farneback Optical Flow completed!")
print("Total frames processed:", frame_count)

if frame_count > 0:
```

```
    print(
        "Average motion:",
        round(total_magnitude / frame_count, 2)
    )
```

```
Farneback Optical Flow completed!
Total frames processed: 312
Average motion: 0.58
```

In [20]:

```
files.download('farneback_output.mp4')
```

```
<IPython.core.display.Javascript object>
```

<IPython.core.display.Javascript object>

In [21]:

```
print("=" * 55)
print("OPTICAL FLOW COMPARISON")
print("=" * 55)

print("\nLucas-Kanade:")
print("- Sparse Optical Flow")
print("- Tracks selected feature points")
print("- Faster and lightweight")
print("- Suitable for feature/object tracking")

print("\nFarneback:")
print("- Dense Optical Flow")
print("- Estimates motion for many pixels")
print("- Gives detailed motion information")
print("- Useful for overall scene motion analysis")

print("\nConclusion:")
print("Lucas-Kanade is generally better for tracking selected")
print("feature points, while Farneback provides dense motion")
print("information across the image.")
```

```
=======================================================
OPTICAL FLOW COMPARISON
=======================================================

Lucas-Kanade:
- Sparse Optical Flow
- Tracks selected feature points
- Faster and lightweight
- Suitable for feature/object tracking

Farneback:
- Dense Optical Flow
- Estimates motion for many pixels
- Gives detailed motion information
- Useful for overall scene motion analysis

Conclusion:
Lucas-Kanade is generally better for tracking selected
feature points, while Farneback provides dense motion
information across the image.
```

In [22]:

```
cap = cv2.VideoCapture('lucas_kanade_output.mp4')

ret, frame = cap.read()

if ret:
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 7))
    plt.imshow(frame_rgb)
    plt.title("Lucas-Kanade Optical Flow - Tracking Result")
    plt.axis("off")
    plt.show()

cap.release()
```

Lucas-Kanade Optical Flow - Tracking Result

